UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Laboratorio 5

Consultas SQL: Andes Bike

Sistemas de información para la gestión (ICN-292)

Profesores: José L. Sáez – Patricio Cifuentes

Clemente Luoni Ortega

Paralelo:100

Rut: 21.830.240-8

Rol: 202360592-1

Fecha: 12/11/2026

Repositorio GitHub:

https://github.com/cluoni/ICN292-Lab5-Luoni-Clemente

1. Consultas

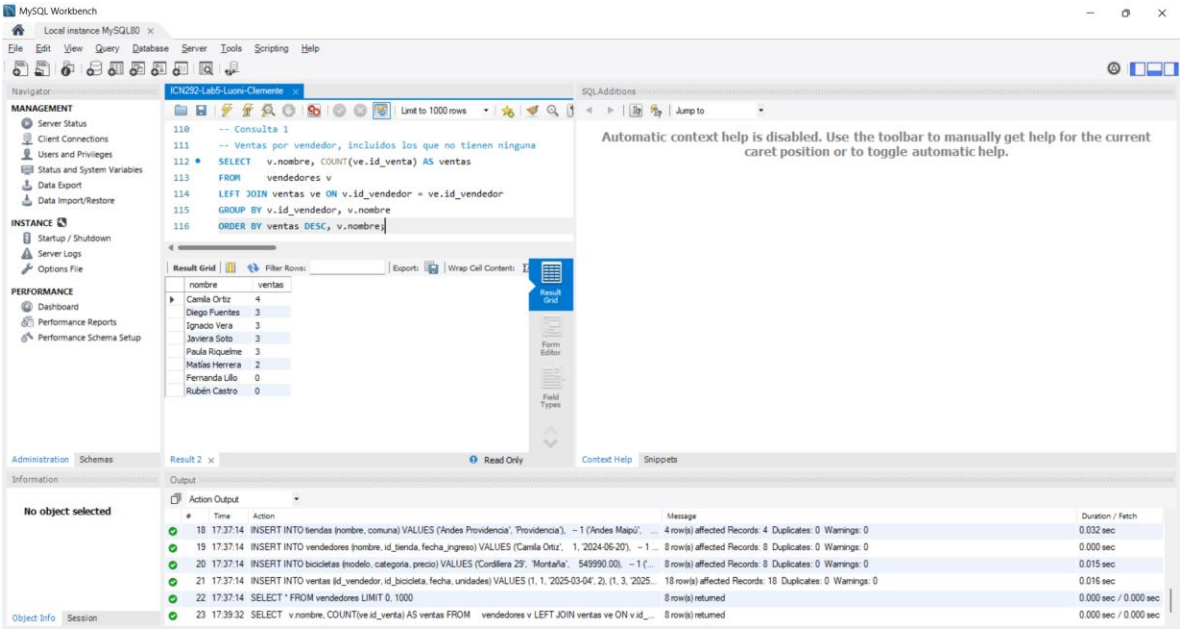
1.1 Consulta 1

Para esta consulta se pidió mostrar a todos los vendedores con la cantidad de ventas que tiene cada uno, incluyendo a los que no han vendido nada. Por eso se usó LEFT JOIN y se contó COUNT(ve.id_venta) en vez de COUNT(*), así los que no tienen ventas salen con 0. Camila Ortiz quedó primera con 4 ventas, y al final aparecen Fernanda Lillo y Rubén Castro con 0.

Código:

```
-- Consulta 1
-- Ventas por vendedor, incluidos los que no tienen ninguna
SELECT   v.nombre, COUNT(ve.id_venta) AS ventas
FROM     vendedores v
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
ORDER BY ventas DESC, v.nombre;
```

Resultado:



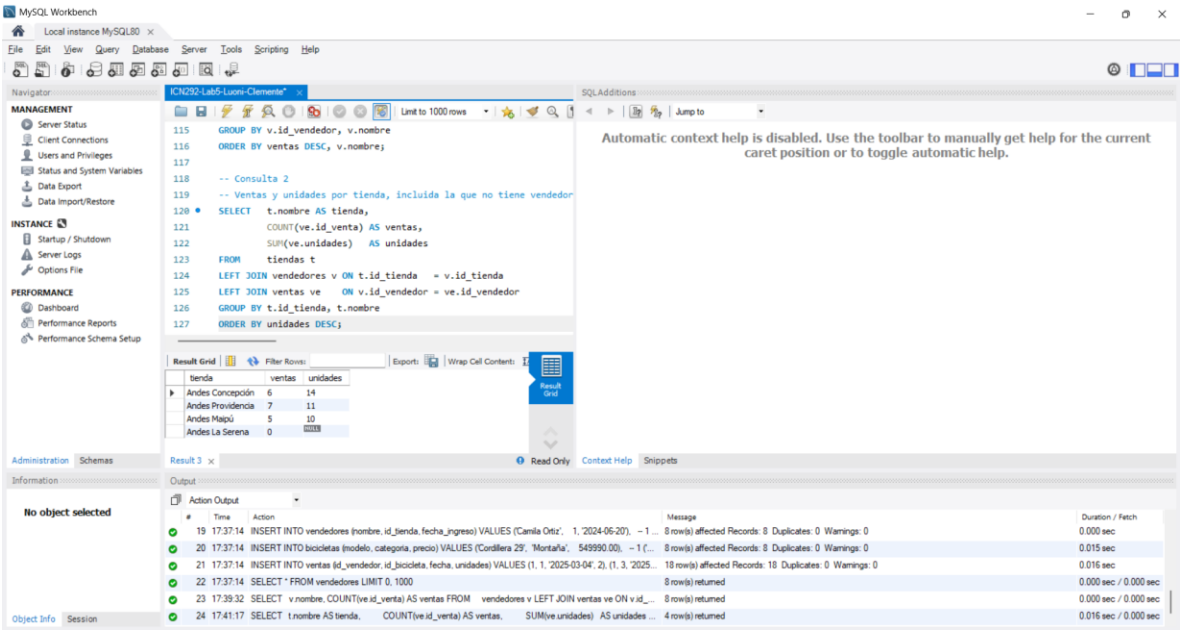
1.2 Consulta 2

Acá se buscan las ventas y las unidades vendidas por cada tienda. Se unieron tiendas, vendedores y ventas con LEFT JOIN para que aparezcan las cuatro tiendas aunque alguna no tenga movimiento. Andes Concepción vendió 14 unidades, Providencia 11 y Maipú 10, mientras que La Serena sale con 0 ventas y NULL en unidades porque no tiene vendedores.

Código:

```
-- Consulta 2
-- Ventas y unidades por tienda, incluida la que no tiene vendedores
SELECT   t.nombre AS tienda,
         COUNT(ve.id_venta) AS ventas,
         SUM(ve.unidades)   AS unidades
FROM     tiendas t
LEFT JOIN vendedores v ON t.id_tienda   = v.id_tienda
LEFT JOIN ventas ve    ON v.id_vendedor = ve.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades DESC;
```

Resultado:



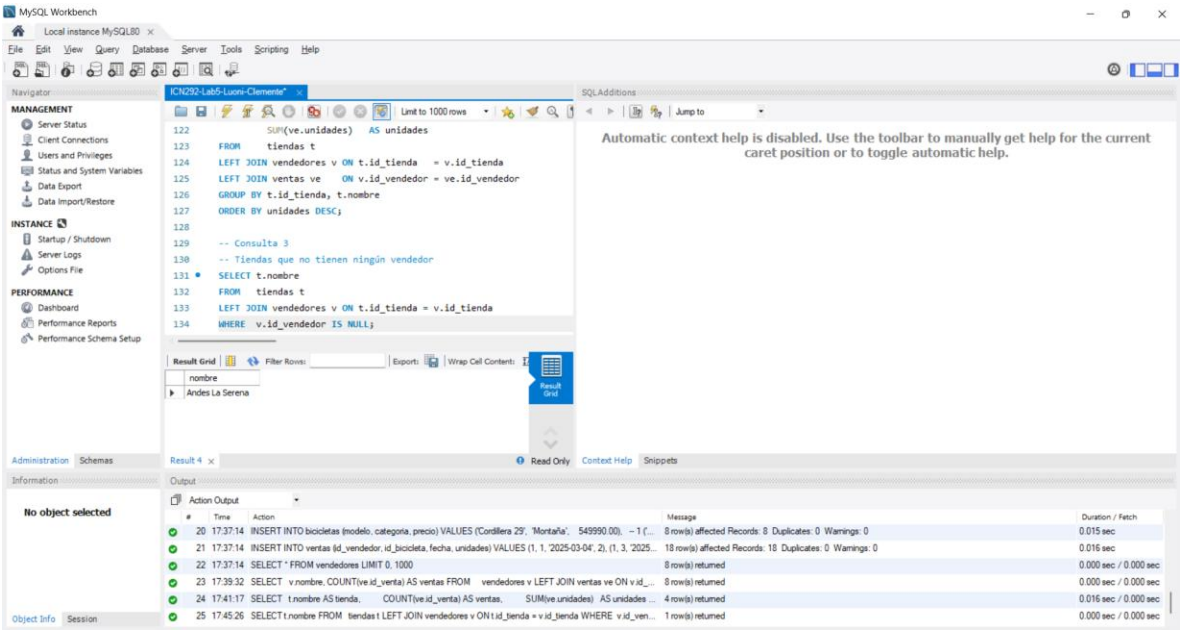
1.3 Consulta 3

Esta consulta muestra las tiendas que no tienen ningún vendedor asignado. Se hizo un LEFT JOIN de tiendas con vendedores y se dejaron solo las filas donde el vendedor quedó en NULL. El único resultado fue Andes La Serena.

Código:

```
-- Consulta 3
-- Tiendas que no tienen ningún vendedor
SELECT t.nombre
FROM   tiendas t
LEFT JOIN vendedores v ON t.id_tienda = v.id_tienda
WHERE  v.id_vendedor IS NULL;
```

Resultado:



1.4 Consulta 4

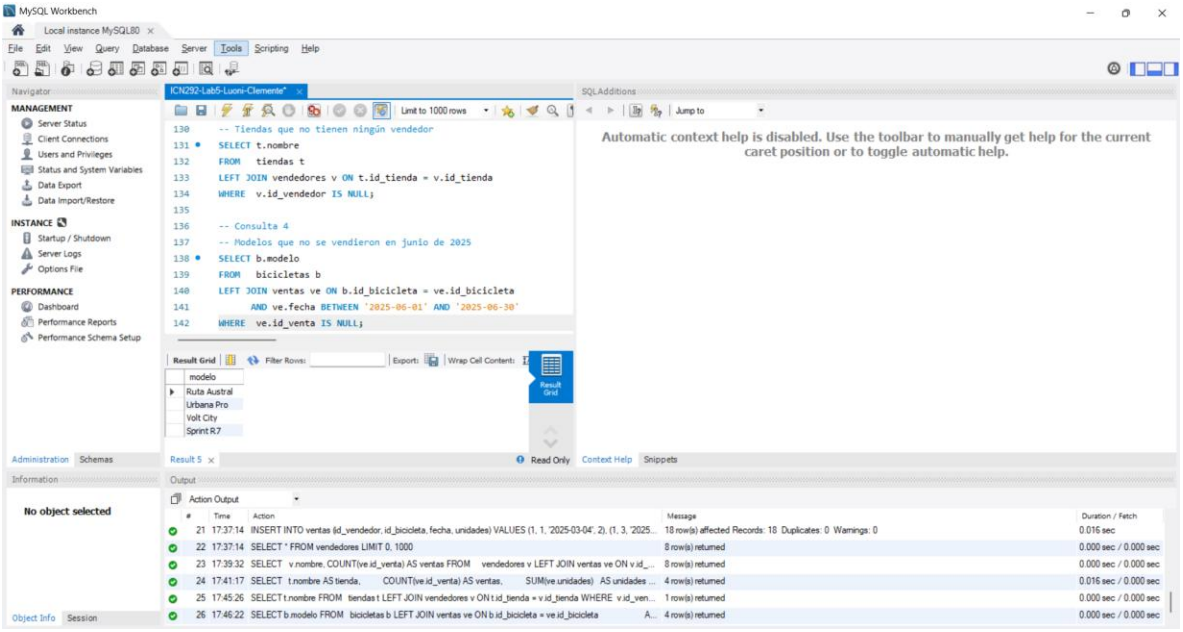
Se pidió listar los modelos de bicicleta que no tuvieron ventas en junio de 2025. Salieron 4 modelos: Ruta Austral, Urbana Pro, Volt City y Sprint R7.

La condición del mes se puso en el ON del LEFT JOIN y no en el WHERE. Si se pone en el WHERE la consulta no devuelve nada, porque estaría pidiendo que la venta sea de junio y al mismo tiempo que sea NULL, y eso no se cumple nunca.

Código:

```
-- Consulta 4
-- Modelos que no se vendieron en junio de 2025
SELECT b.modelo
FROM   bicicletas b
LEFT JOIN ventas ve ON b.id_bicicleta = ve.id_bicicleta
        AND ve.fecha BETWEEN '2025-06-01' AND '2025-06-30'
WHERE  ve.id_venta IS NULL;
```

Resultado:



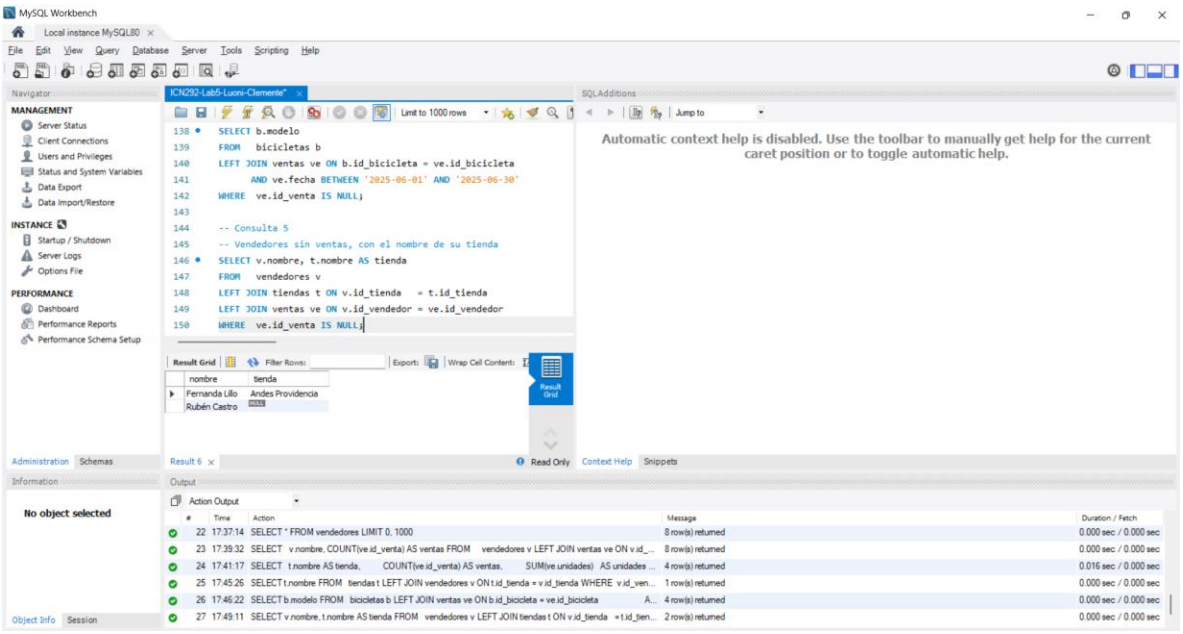
1.5 Consulta 5

En este caso se buscan los vendedores que no tienen ventas, junto con el nombre de su tienda. Los dos joins son LEFT JOIN, porque si el de tiendas fuera un JOIN normal Rubén Castro quedaría fuera al no tener tienda. Los resultados fueron Fernanda Lillo, de Andes Providencia, y Rubén Castro, que sale con la tienda en NULL.

Código:

```
-- Consulta 5
-- Vendedores sin ventas, con el nombre de su tienda
SELECT v.nombre, t.nombre AS tienda
FROM   vendedores v
LEFT JOIN tiendas t ON v.id_tienda   = t.id_tienda
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
WHERE  ve.id_venta IS NULL;
```

Resultado:



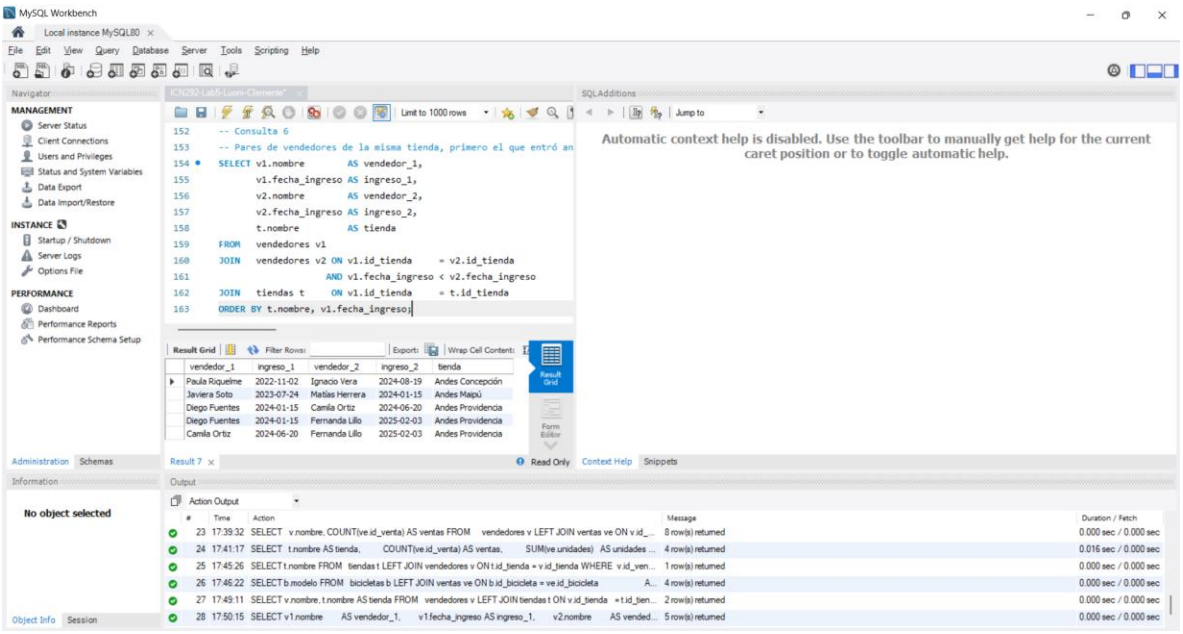
1.6 Consulta 6

Para esta consulta se pidieron los pares de vendedores que trabajan en la misma tienda. Para eso se unió la tabla vendedores consigo misma, y con la condición de que la fecha de ingreso del primero sea menor a la del segundo cada par aparece una sola vez y primero el que entró antes. Salieron 5 pares: tres de Providencia, uno de Maipú y uno de Concepción.

Código:

```
-- Consulta 6
-- Pares de vendedores de la misma tienda, primero el que entró antes
SELECT v1.nombre        AS vendedor_1,
       v1.fecha_ingreso AS ingreso_1,
       v2.nombre        AS vendedor_2,
       v2.fecha_ingreso AS ingreso_2,
       t.nombre         AS tienda
FROM   vendedores v1
JOIN   vendedores v2 ON v1.id_tienda     = v2.id_tienda
                    AND v1.fecha_ingreso < v2.fecha_ingreso
JOIN   tiendas t     ON v1.id_tienda     = t.id_tienda
ORDER BY t.nombre, v1.fecha_ingreso;
```

Resultado:



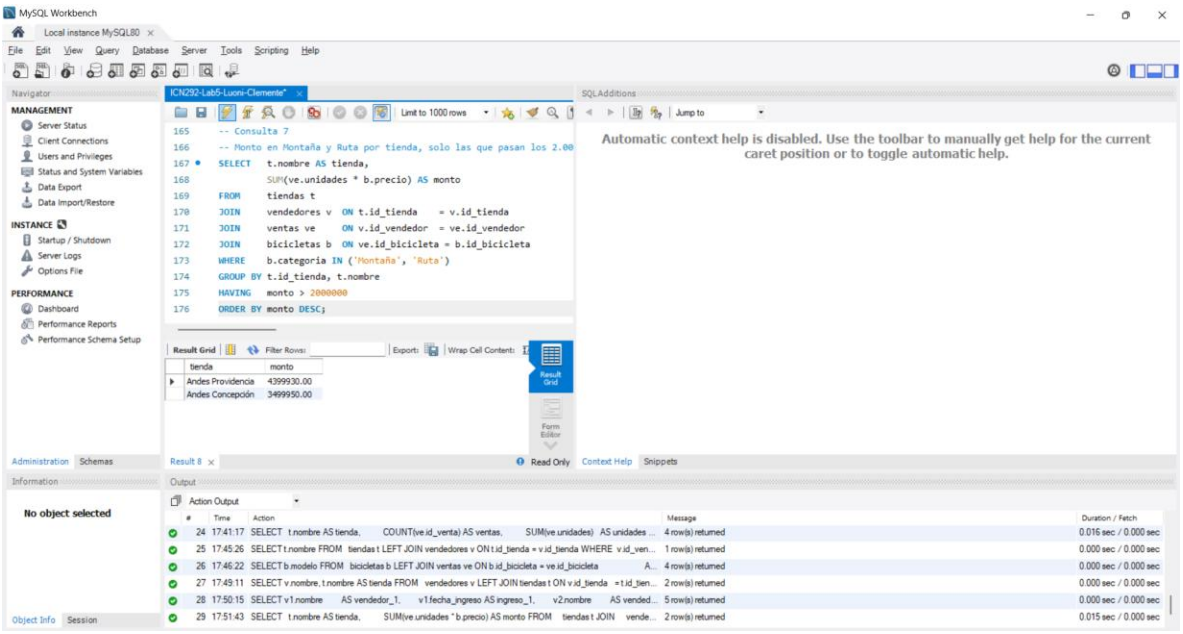
1.7 Consulta 7

Esta consulta calcula el monto vendido en bicicletas de montaña y ruta por cada tienda, dejando solo las que superan los $2.000.000. Se unieron las cuatro tablas, se filtró la categoría en el WHERE y el monto en el HAVING, ya que es un valor agrupado. Quedaron Andes Providencia con $4.399.930 y Andes Concepción con $3.499.950.

Código:

```
-- Consulta 7
-- Monto en Montaña y Ruta por tienda, solo las que pasan los 2.000.000
SELECT   t.nombre AS tienda,
         SUM(ve.unidades * b.precio) AS monto
FROM     tiendas t
JOIN     vendedores v  ON t.id_tienda    = v.id_tienda
JOIN     ventas ve     ON v.id_vendedor  = ve.id_vendedor
JOIN     bicicletas b  ON ve.id_bicicleta = b.id_bicicleta
WHERE    b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING   monto > 2000000
ORDER BY monto DESC;
```

Resultado:



1.8 Consulta 8

Se pidió mostrar las bicicletas que tienen un precio mayor al promedio del catálogo, ordenadas de mayor a menor. Primero se calculó el promedio por separado y después se hizo la consulta. Las bicicletas que lo superan son Volt City, Sprint R7, Ruta Austral y Trail Master.

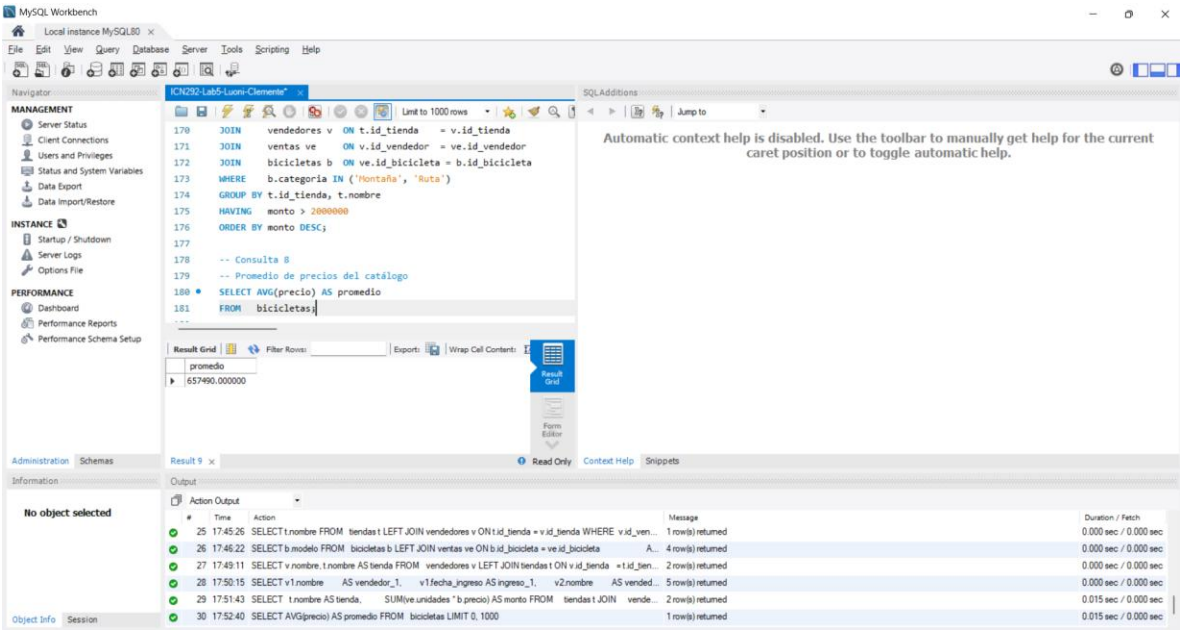
El promedio del catálogo es $657.490 y se obtuvo con la subconsulta SELECT AVG(precio) FROM bicicletas, que promedia los precios de los 8 modelos y se usa dentro del WHERE para comparar cada precio.

Código:

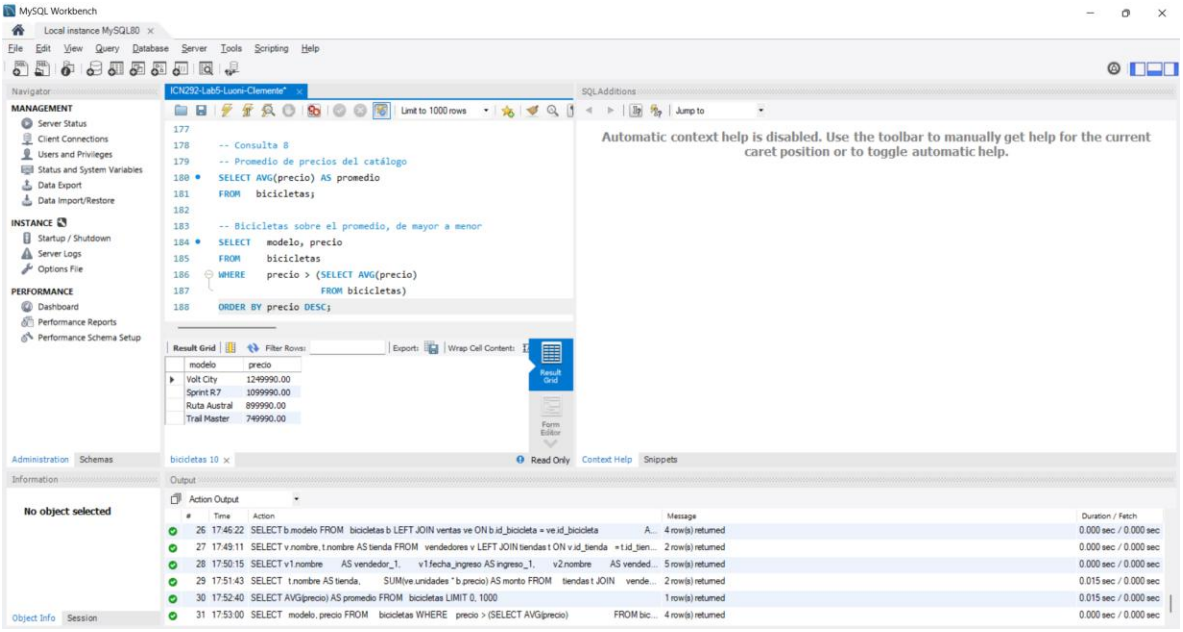
```
-- Consulta 8
-- Promedio de precios del catálogo
SELECT AVG(precio) AS promedio
FROM   bicicletas;


-- Bicicletas sobre el promedio, de mayor a menor
SELECT   modelo, precio
FROM     bicicletas
WHERE    precio > (SELECT AVG(precio)
                   FROM bicicletas)
ORDER BY precio DESC;
```

Resultado (promedio del catálogo):



Resultado (bicicletas sobre el promedio):



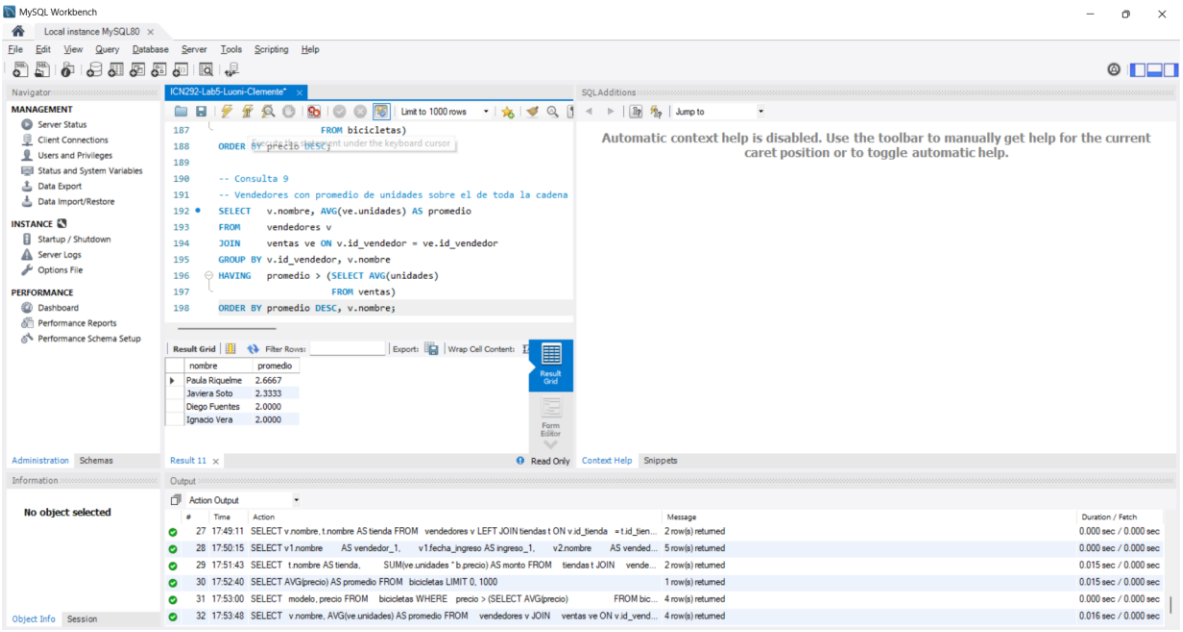
1.9 Consulta 9

Acá se buscan los vendedores cuyo promedio de unidades por venta es mayor al promedio de toda la cadena, que es 1,94. El promedio de cada vendedor se calcula con AVG y se compara en el HAVING con una subconsulta. Salieron Paula Riquelme con 2,67, Javiera Soto con 2,33 y Diego Fuentes e Ignacio Vera con 2.

Código:

```
-- Consulta 9
-- Vendedores con promedio de unidades sobre el de toda la cadena
SELECT   v.nombre, AVG(ve.unidades) AS promedio
FROM     vendedores v
JOIN     ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
HAVING   promedio > (SELECT AVG(unidades)
                     FROM ventas)
ORDER BY promedio DESC, v.nombre;
```

Resultado:



1.10 Consulta 10

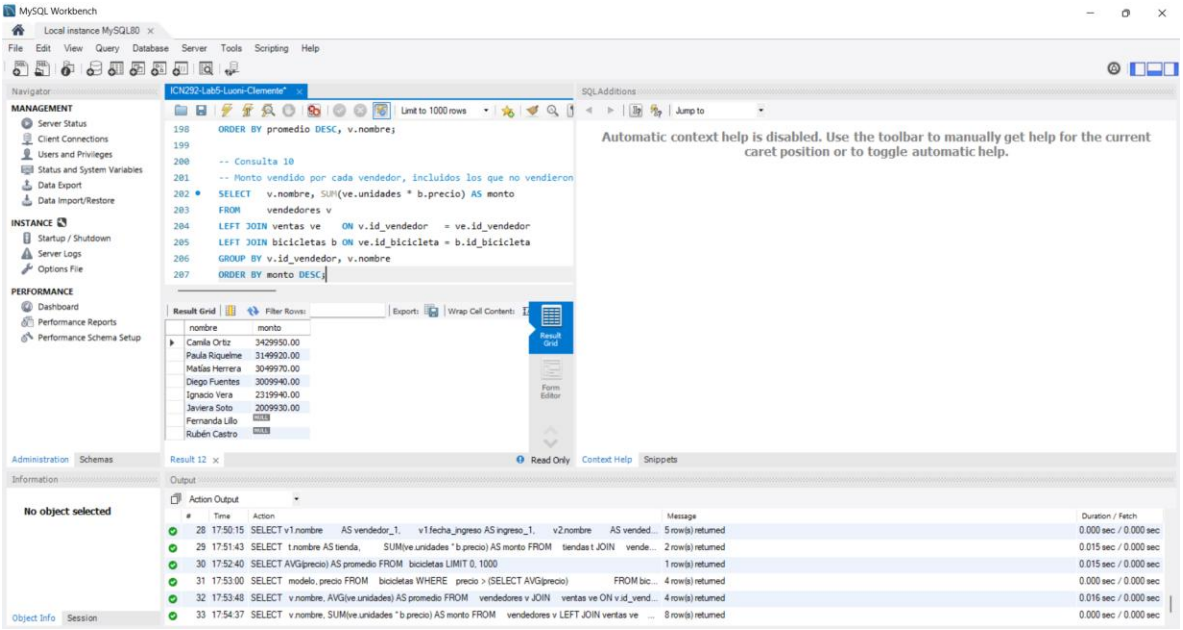
Esta consulta muestra el monto total vendido por cada vendedor, multiplicando las unidades por el precio de la bicicleta, e incluyendo también a los que no vendieron nada. Camila Ortiz es la que más vendió con $3.429.950 y Javiera Soto la que menos de los que sí tienen ventas, con $2.009.930.

Fernanda Lillo y Rubén Castro aparecen con NULL y no con cero porque no tienen ventas, entonces SUM no tiene nada que sumar y devuelve NULL. Para que salieran con cero habría que usar COALESCE(SUM(ve.unidades * b.precio), 0).

Código:

```
-- Consulta 10
-- Monto vendido por cada vendedor, incluidos los que no vendieron
SELECT   v.nombre, SUM(ve.unidades * b.precio) AS monto
FROM     vendedores v
LEFT JOIN ventas ve    ON v.id_vendedor   = ve.id_vendedor
LEFT JOIN bicicletas b ON ve.id_bicicleta = b.id_bicicleta
GROUP BY v.id_vendedor, v.nombre
ORDER BY monto DESC;
```

Resultado:



2. Conclusión

Finalmente, se logró crear la base de Andes Bike con sus cuatro tablas y resolver las diez consultas, comprobando cada una con su captura.

Lo que más sirvió de este laboratorio fue entender bien el LEFT JOIN, porque en varias consultas los datos importantes eran justamente los que no tenían pareja, como La Serena sin vendedores o Fernanda y Rubén sin ventas. También quedó clara la diferencia entre poner una condición en el ON o en el WHERE, y que SUM devuelve NULL cuando no hay filas que sumar.